

Topic 3

Intersystem communication

Agenda

- Overview of the previous lecture
- Introduction
- Distributed applications
 1. Distributed applications: Features
 2. Distributed applications: Architectures
 3. Distributed applications: Implementation (simulation on a single computer)
 4. Important resources
- ~~Network programming~~
 1. ~~Sockets and streams~~
 2. ~~Simple client-server application~~
 3. ~~Application to access URL~~
 4. ~~Important resources~~
- ~~CORBA~~
- Remote Message Invocation
 1. Features
 2. Example: in Java
- ~~Web services~~
 1. ~~Features~~
 2. ~~Example: implementation in Java~~
- ~~Messaging services~~
- Summary

Introduction

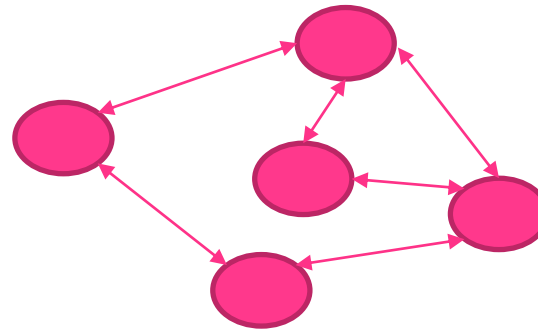
❑ What is Intersystem communication

1. Distributed hardware and software **components** communicate to perform business processes (or functions)
2. Intersystem communications is, therefore, about **distributed components** and how they communicate/**work together**
3. Two types: distributed **hardware** and distributed **application**
4. We'll focus on distributed applications
 1. These are **separate** and **independent** application which communicate
 2. An application is generally **standalone** (i.e., processes and data) and **focuses on a business process**

❑ **Part 1: Distributed applications**

Introduction

- ❑ Distributed applications consist of different applications which:
 1. Communicate over a network
 2. Share data, resources and processes to achieve a common goal



- ❑ Example
 - ❑ Online shopping application: Order, Delivery, Payment, Returns, Inventory, Recommender
 - ❑ Banking: Customer service, Transactions, Communication and
 - Communication and information sharing (between the different applications)

Introduction

- ❑ Why the growth in distributed applications?
 1. Networks and the Internet
 2. Distributed architectures (i.e., design patterns for distributed applications)
 3. Modularity: reusable components:
 - Can you explain why OOP is important to distributed applications ?
 4. Open source

Introduction

❑ Advantages of distributed applications

1. Shared resource
2. Scalability: easier to add more components
3. Improved performance: distributed work load
4. Reliability: fault tolerance (other parts are in use when some parts go down)
5. Manageability: “smaller” parts are easier to manage
6. Etc.

❑ Disadvantages: managing too many smaller applications can be challenging

Distributed applications: **Features**

1. **Independent** (each application with local processing and data)
2. **Data sharing** among applications
3. **Distributed processing** is (i.e., independent processes)
4. **Multithreading** to control independent processes
5. **Distributed objects** (i.e., components or parts of the application are distributed)

Distributed applications: **Features**

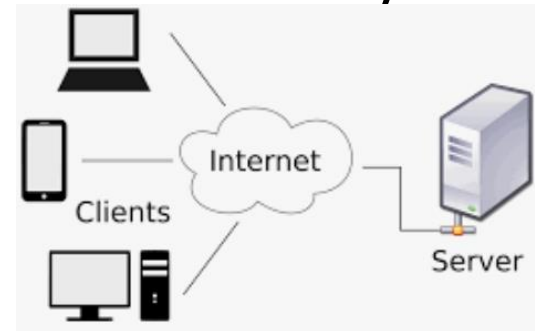
- 6. Networks** for communication
- 7. Communication protocols** (why protocols?
Interoperability)
 - a. Low-level (e.g., TCP/IP)
 - b. High-level (e.g., format of information shared)
- 8. Message queuing** (e.g., to backup messages during communications failure)
- 9. Exceptions handling** (why?)
- 10. Security** (why?)
 - a. Authentication
 - b. Authorization
 - c. Etc.

Distributed applications

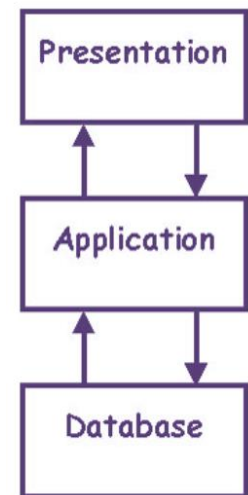
Question: briefly explain why application developers need to be familiar with distributed architectures.

Distributed applications: **Architectures**

- ❑ **Client-server architecture:** separates the **server** (i.e., information processing as a group of services) and the **clients** (i.e., users of the service)



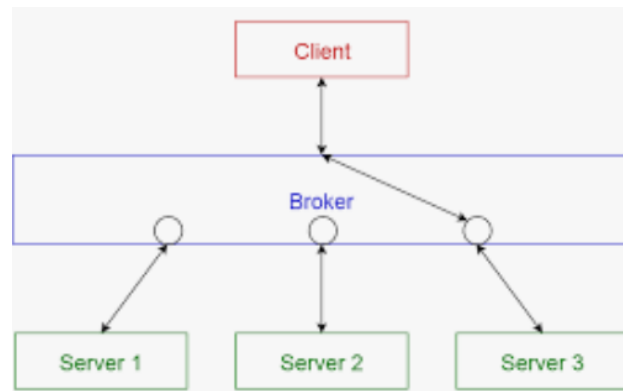
- ❑ **N-tier architecture:** consists of several servers, each with its own clients
 - The most popular is the **3-tier architecture:** separates information processing (i.e., application from data management and presentation (i.e., display)



Distributed applications: **Architectures**

❑ **Broker architecture:** consists of the **client**, **broker** and the **server**

1. The **broker** coordinates interaction between the client and server
2. There is no direct communication between the client and the server (i.e., interaction is by **remote service calls**)
3. Server registers a service with the broker
4. Client requests a service
5. The broker directs the request to the appropriate service in the registry and returns response to the client



Distributed applications: **Architectures**

❑ **Most popular broker architectures**

1. Common Object Request Broker Architecture (CORBA)
 - 1st generation
 - Industry standard (i.e., platform independent)
2. Remote Method Invocation
 - 2nd generation
 - Language dependent (i.e., implemented in Java, C#, etc.)
3. Web services and Micro services

❑ CORBA and RMI have been overtaken by modern technologies including above

Distributed applications: **Architectures**

RMI is Dead, but Textbooks and Colleges shouldn't Stop Teaching it

It's easy to understand distributed system architecture if you are familiar with RMI.

[By Janaka Ekanayake](#)

Distributed applications: Architectures

❑ Service oriented Architecture (SOA)

- Combines the client-server and broker architectures
- Usually implemented as [Web service](#)
- 1. A **service** are standalone (i.e., encapsulates code and data for a standalone business function
- 2. Services are:
 - 1. Loosely coupled)
 - 2. Reusable
 - 3. Platform-independent, allowing interoperability)
 - 4. Scalable ([why?](#))
 - 5. **Any other advantages? Disadvantages?**
- 3. Example: a bank [loan application](#) may consist of ff. services:
 - 1. [Check customer's credit score](#)
 - 2. [Calculate monthly loan repayment](#)
 - 3. [Process application](#)
 - 4. Etc.
- ❑ **Question:** have you noticed the move from distributed objects to [services](#) in distributed architectures?

Distributed applications: SOA

❑ Service oriented Architecture (SOA)

1. SOA is based on standards, e.g., for:
 1. Messaging
 2. Data (i.e., to structure and share data)
 3. Network
 4. Interface
 5. Question: what are the benefits of standards above
2. Service interfaces
 1. Are **contracts** between a service provider and service consumer
 2. Reduce coupling and increases cohesion
 3. May be:
 1. Written in a programming language
 2. Written Supplied by a vendor (e.g., IBM SAP)
 - Web services use **WSDL (Web Services Definition Language)** based on XML

Distributed applications: Architectures

❑ Web services

1. Are services accessible across networks
2. Services are:
 1. Published in a registry
 2. Exposed to consumers
 3. Consumed by consumers
3. Two main architectures:
 1. **SOAP** (Simple Object Access Protocol)
 - Based on Service Oriented Architecture: **SOA**
 2. **REST** (Representational State Transfer)
 - Based on Resource Oriented Architecture: **ROA**
4. Each has its own standards for:
 1. Messaging
 2. Data
 3. Network
 4. Interface
 5. Etc.
5. E.g., standards for SOAP:
 1. Messaging: XML-based protocol
 2. Data: XML (to structure and share data)
 3. Network: **http**
 4. Interface (e.g., using **WSDL** (Web Services Definition Language) based on **XML**)

Distributed applications: **Implementation**

- ❑ We have discussed the following:
 1. Client server
 2. Broker
 3. Service oriented
- ❑ Architectures (or patterns) for distributed applications have evolved, from:
 1. Distributed object architectures to Service oriented architectures and Web services
- ❑ Modern architectures include (self-reading)
 1. Micro services
 2. Resource oriented service

Distributed applications: **Implementation**

- ❑ We have discussed some of the architectures (**patterns**) for distributed applications
- ❑ So what next?
 1. **Implementation**: once you understand the patterns, you can adopt a specific pattern to write distributed application
 2. We will write distributed application

Summary

- ❑ Distributed architectures have evolved from
 1. Distributed object architectures to
 2. Service oriented architectures
 3. Web services
- ❑ Modern architectures include
 1. Resource oriented service
 2. Micro services
- ❑ **Exercise:** read about the following architectures
 1. Microservice and
 2. Resource oriented

Comments and feedback

?

Next.....

- ~~Intersystem communication: Part 2 (Network programming) with Sockets~~